

Arithmetic

C

```
float add(float a, float b) {
    return a + b;
}

float subtract(float lhs,
               float rhs) {
    return lhs - rhs;
}

float multiply(float a, float b) {
    return a * b;
}

float divide(float lhs,
             float rhs) {
    return lhs / rhs;
}
```

LOWERED MLIR

```
module attributes {cir.triple = "x86_64-unknown-linux-gnu"} {
  func.func @add(%arg0: f32, %arg1: f32) -> f32 {
    %0 = arith.addf %arg0, %arg1 : f32
    return %0 : f32
  }
  func.func @subtract(%arg0: f32, %arg1: f32) -> f32 {
    %0 = arith.subf %arg0, %arg1 : f32
    return %0 : f32
  }
  func.func @multiply(%arg0: f32, %arg1: f32) -> f32 {
    %0 = arith.mulf %arg0, %arg1 : f32
    return %0 : f32
  }
  func.func @divide(%arg0: f32, %arg1: f32) -> f32 {
    %0 = arith.divf %arg0, %arg1 : f32
    return %0 : f32
  }
}
```

Counted loop

C

```
float accumulate(float x) {
    float result = 1.0f;
    for (int i = 0; i < 4; ++i)
        result += x;
    return result;
}
```

LOWERED MLIR

```
module attributes {cir.triple = "x86_64-unknown-linux-gnu"}
{
  func.func @accumulate(%arg0: f32) -> f32 {
    %c1_i32 = arith.constant 1 : i32
    %c0_i32 = arith.constant 0 : i32
    %c4_i32 = arith.constant 4 : i32
    %cst = arith.constant 1.000000e+00 : f32
    %0 = scf.for %arg1 = %c0_i32 to %c4_i32 step %c1_i32
      iter_args(%arg2 = %cst) -> (f32) : i32 {
        %1 = arith.addf %arg2, %arg0 : f32
        scf.yield %1 : f32
      }
    return %0 : f32
  }
}
```

Loop with an if

C

```
float conditional_sum(float x) {
    float result = 0.0f;
    for (int i = 0; i < 4; ++i) {
        if (i < 2)
            result += x;
    }
    return result;
}
```

LOWERED MLIR

```
module attributes {cir.triple = "x86_64-unknown-linux-gnu"} {
  func.func @conditional_sum(%arg0: f32) -> f32 {
    %c1_i32 = arith.constant 1 : i32
    %c0_i32 = arith.constant 0 : i32
    %c2_i32 = arith.constant 2 : i32
    %c4_i32 = arith.constant 4 : i32
    %cst = arith.constant 0.000000e+00 : f32
    %0 = scf.for %arg1 = %c0_i32 to %c4_i32 step %c1_i32
      iter_args(%arg2 = %cst) -> (f32) : i32 {
        %1 = arith.cmpi slt, %arg1, %c2_i32 : i32
        %2 = scf.if %1 -> (f32) {
          %3 = arith.addf %arg2, %arg0 : f32
          scf.yield %3 : f32
        } else {
          scf.yield %arg2 : f32
        }
        scf.yield %2 : f32
      }
    return %0 : f32
  }
}
```

Current limitations

Branch merges

Some merged values need an explicit `set_range` before further TAFFO arithmetic.

Nested loops

TAFFO underestimates the accumulator range in our nested-loop example.

Conditions inside loops

TAFFO range analysis times out on the tested conditional loop.

Runtime bounds

TAFFO estimates the iteration count when the loop bound is unknown.

General while loops

The tested condition-driven while loop does not complete the TAFFO pipeline.